

EE6406

Analytic and Ensemble Machine Learning

(Lecture 5: Over- and Under-determined Analytic Regression)

Kar-Ann Toh

School of Electrical and Electronic Engineering
Nanyang Technological University

karanntoh@ntu.edu.sg

Semester 1, AY2026-27

EE6406: Analytic and Ensemble Machine Learning

Lecture 5

Over- and Under-determined Analytic Regression

Overview

- 1 Introduction
- 2 Regression Error Based Learning Methods
 - SSE Minimization for Over-determined Systems
 - SSE Minimization for Under-determined Systems
 - Primal Ridge Regression
 - Dual Ridge Regression
 - Kernel Regression
 - Kernel Ridge Regression
 - Threshold and Classification Decision

Introduction

The main goal of **learning** is to make use of the available training samples, **to form an inference engine for prediction**.

The underlying assumption is thus hinged upon the fidelity of **unseen data's distribution** with respect to the **training data's distribution**.

In general, those learning score functions discussed in the previous lecture can be adopted to form the cost or criterion functions for regression predictor or classifier learning. We shall focus on several foundation **analytic learning and estimation methods** in this lecture.

- Consider a **binary or two-category classification problem**. Given a training data set which consists of m examples $(\mathbf{x}_i, y_i)$, $i = 1, \dots, m$, where $\mathbf{x}_i \in \mathbb{R}^d$ denotes the i^{th} sample of the feature vector, and $y_i \in \{0, 1\}$ (or $y_i \in \{-1, +1\}$) denotes the corresponding *class indicator* or *target label*. In other words, the value y_i can be viewed as the class associated with $\mathbf{x}_i$.
- Utilizing this training set, the goal is **to determine a predictor $g(\mathbf{x}, \hat{\mathbf{w}})$ at a weight setting $\hat{\mathbf{w}}$ and a threshold value $\hat{\tau}$ such that a correct class prediction can be obtained for novel feature samples $\mathbf{x}_j$ not found in the training set.**
- An ideal classifier is such that $cls(g(\mathbf{x}_j, \hat{\mathbf{w}}) \geq \tau) = y_j$ for all $j = 1, 2, \dots, n$, which relates each unseen data $\mathbf{x}_j$ to its true target label y_j .

$$J_{\text{SSE}}(\mathbf{w}) = \sum_{i=1}^m (y_i - g(\mathbf{x}_i, \mathbf{w}))^2. \quad (5.1)$$
$$g(\mathbf{x}_i, \mathbf{w}) = \mathbf{w}^T \mathbf{p}(\mathbf{x}_i), \quad i = 1, 2, \dots, m, \quad (5.2)$$

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

$$J_{\text{SSE}}(\mathbf{w}) = \|\mathbf{y} - \mathbf{P}\mathbf{w}\|_2^2 = (\mathbf{y} - \mathbf{P}\mathbf{w})^T (\mathbf{y} - \mathbf{P}\mathbf{w}), \quad (5.3)$$
$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}, \mathbf{P} = \begin{bmatrix} 1 & p_1(\mathbf{x}_1) & \cdots & p_D(\mathbf{x}_1) \\ 1 & p_1(\mathbf{x}_2) & \cdots & p_D(\mathbf{x}_2) \\ \vdots & \vdots & \ddots & \vdots \\ 1 & p_1(\mathbf{x}_m) & \cdots & p_D(\mathbf{x}_m) \end{bmatrix}, \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_D \end{bmatrix}. \quad (5.4)$$

$$-2\mathbf{P}^T(\mathbf{y} - \mathbf{P}\mathbf{w}) = \mathbf{0}. \quad (5.5)$$
$$\hat{\mathbf{w}} = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{y}. \quad (5.6)$$

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

$$g(\mathbf{x}_i, \hat{\mathbf{w}}) = \hat{\mathbf{w}}^T \mathbf{p}(\mathbf{x}_i), \quad i = 1, 2, \dots, m. \quad (5.7)$$

The entire set of training input data can be stacked in matrix form as $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \cdots \mathbf{x}_m]^T$ and the corresponding stacked transformed feature matrix is $\mathbf{P}$ in (5.4) where the trained output can be stacked as

$$g(\mathbf{X}, \hat{\mathbf{w}}) = \mathbf{P}\hat{\mathbf{w}}. \quad (5.8)$$

For testing or prediction of unseen data given by $\mathbf{x}_j, j = 1, 2, \dots, n$, the prediction output can be obtained by projecting $\mathbf{p}(\mathbf{x}_j)$ onto the learned $\hat{\mathbf{w}}$ giving

$$g(\mathbf{x}_j, \hat{\mathbf{w}}) = \hat{\mathbf{w}}^T \mathbf{p}(\mathbf{x}_j), \quad j = 1, 2, \dots, n. \quad (5.9)$$

The stacked form of the above formulation is given by

$$g(\mathbf{X}_t, \hat{\mathbf{w}}) = \mathbf{P}_t \hat{\mathbf{w}}. \quad (5.10)$$

where the unseen test data set $\mathbf{x}_j, j = 1, 2, \dots, n$ has been stacked in $\mathbf{P}_t$ as the transformed test feature matrix. For classification, a threshold process is followed.

$$\mathbf{G} = \mathbf{P}\mathbf{W}, \quad (5.11)$$
$$\begin{aligned} \mathbf{G} &= [g(\mathbf{x}, \mathbf{w}_1), g(\mathbf{x}, \mathbf{w}_2), \dots, g(\mathbf{x}, \mathbf{w}_C)] \\ &= \begin{bmatrix} g(\mathbf{x}_1, \mathbf{w}_1) & g(\mathbf{x}_1, \mathbf{w}_2) & \dots & g(\mathbf{x}_1, \mathbf{w}_C) \\ g(\mathbf{x}_2, \mathbf{w}_1) & g(\mathbf{x}_2, \mathbf{w}_2) & \dots & g(\mathbf{x}_2, \mathbf{w}_C) \\ \vdots & \dots & \ddots & \vdots \\ g(\mathbf{x}_m, \mathbf{w}_1) & g(\mathbf{x}_m, \mathbf{w}_2) & \dots & g(\mathbf{x}_m, \mathbf{w}_C) \end{bmatrix}, \end{aligned} \quad (5.12)$$

$$\begin{aligned} \mathbf{W} &= [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_C] \\ &= \begin{bmatrix} w_{0,1} & w_{0,2} & \cdots & w_{0,C} \\ w_{1,1} & w_{1,2} & \cdots & w_{1,C} \\ \vdots & & \ddots & \vdots \\ w_{D,1} & w_{D,2} & \cdots & w_{D,C} \end{bmatrix}, \end{aligned} \quad (5.13)$$

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

$$\begin{aligned} J_{\text{SSE}}(\mathbf{W}) &= \text{trace}\{(\mathbf{Y} - \mathbf{G})^T(\mathbf{Y} - \mathbf{G})\} \\ &= \text{trace}\{(\mathbf{Y} - \mathbf{PW})^T(\mathbf{Y} - \mathbf{PW})\}, \end{aligned} \quad (5.14)$$
$$\mathbf{Y} = \left[\begin{array}{c} \mathbf{Y}_1 \\ \mathbf{Y}_2 \\ \vdots \\ \mathbf{Y}_C \end{array} \right] = \left[\begin{array}{cccc} \overbrace{1 & 0 & \dots & 0}^C \\ 1 & 0 & \dots & 0 \\ \vdots & \vdots & \dots & \vdots \\ 0 & 1 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & & & \vdots \\ 0 & 0 & \ddots & 1 \\ 0 & 0 & \dots & 1 \end{array} \right] \left\{ \begin{array}{l} \mathbf{Y}_1 \\ \mathbf{Y}_2 \\ \vdots \\ \mathbf{Y}_C \end{array} \right\} m. \quad (5.15)$$

- In the above illustration, $\mathbf{Y}$ consists of stacks of block matrices $\mathbf{Y}_k$, $k = 1, 2, \dots, C$ corresponding to each class and all of the elements of $\mathbf{Y}_k$ are zero except for those in the k th column, which are unity. This is assuming the corresponding sample rows of the transformed feature matrix $\mathbf{P}$ have been stacked a similar fashion with corresponding category blocks.
- In general, each sample row of $\mathbf{P}$ drawn from a particular category just need to be labelled by the corresponding sample row of $\mathbf{Y}$. In other words, **only row correspondence is needed between $\mathbf{P}$ and $\mathbf{Y}$ and no sorting of stacked blocks is required.**

$$\begin{aligned} \text{trace}\{(\mathbf{Y} - \mathbf{P}\mathbf{W})^T(\mathbf{Y} - \mathbf{P}\mathbf{W})\} &= \text{trace}\{\mathbf{E}^T\mathbf{E}\} \\ &= \text{trace}\left(\begin{bmatrix} \mathbf{e}_1^T \\ \vdots \\ \mathbf{e}_C^T \end{bmatrix} [\mathbf{e}_1, \dots, \mathbf{e}_C]\right) \\ &= \sum_{k=1}^C \mathbf{e}_k^T \mathbf{e}_k, \end{aligned} \quad (5.16)$$

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

$$[\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_2, \dots, \hat{\mathbf{w}}_C] = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T [\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_C]. \quad (5.17)$$
$$\hat{\mathbf{W}} = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{Y}, \quad (5.18)$$

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

$$\hat{\mathbf{G}} = \mathbf{P}\hat{\mathbf{W}} \quad (5.19)$$
$$\hat{\mathbf{G}}_t = \mathbf{P}_t \hat{\mathbf{W}}. \quad (5.20)$$

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

Given the following samples of data features (x_1, x_2) and their corresponding target outputs (y_1, y_2, y_3) for training.

[illegible]

We want to predict the outputs of $(x_1, x_2) = (-1, 3.3)$ and $(x_1, x_2) = (2.5, 0.5)$ using a full polynomial model of second order.

Example 5.1(a) Cont'd

The first step is to pack the data in matrix form:

$$\mathbf{X} = \begin{bmatrix} -5.5 & 6.8 \\ -4.5 & 5.2 \\ -3.3 & 4.6 \\ -2.5 & 3.2 \\ -1.3 & 2.8 \\ -0.5 & 2.1 \\ 0.2 & 1.9 \\ 1.5 & 1.6 \\ 2.8 & 1.3 \\ 3.3 & 0.8 \end{bmatrix}, \quad \mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

With the second-order polynomial expansion given by $\mathbf{p} = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$, the data matrix $\mathbf{X}$ is next transformed into the polynomial matrix given by

$$\mathbf{P} = \begin{bmatrix} 1.0000 & -5.5000 & 6.8000 & 30.2500 & -37.4000 & 46.2400 \\ 1.0000 & -4.5000 & 5.2000 & 20.2500 & -23.4000 & 27.0400 \\ 1.0000 & -3.3000 & 4.6000 & 10.8900 & -15.1800 & 21.1600 \\ 1.0000 & -2.5000 & 3.2000 & 6.2500 & -8.0000 & 10.2400 \\ 1.0000 & -1.3000 & 2.8000 & 1.6900 & -3.6400 & 7.8400 \\ 1.0000 & -0.5000 & 2.1000 & 0.2500 & -1.0500 & 4.4100 \\ 1.0000 & 0.2000 & 1.9000 & 0.0400 & 0.3800 & 3.6100 \\ 1.0000 & 1.5000 & 1.6000 & 2.2500 & 2.4000 & 2.5600 \\ 1.0000 & 2.8000 & 1.3000 & 7.8400 & 3.6400 & 1.6900 \\ 1.0000 & 3.3000 & 0.8000 & 10.8900 & 2.6400 & 0.6400 \end{bmatrix}.$$

The system is over-determined since there are more samples than features. The training is performed using (5.18) to give

$$\hat{\mathbf{W}} = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{Y} = \begin{bmatrix} 0.2999 & -0.3268 & 1.0268 \\ -0.8781 & 1.8294 & -0.9512 \\ -0.4796 & 1.6369 & -1.1572 \\ 0.1717 & -0.3934 & 0.2217 \\ 0.3572 & -0.8690 & 0.5118 \\ 0.1579 & -0.4619 & 0.3040 \end{bmatrix}.$$

$$\hat{\mathbf{G}} = \mathbf{P}_t \hat{\mathbf{W}} = \begin{bmatrix} 0.3080 & 0.6892 & 0.0028 \\ -0.5763 & 1.4049 & 0.1714 \end{bmatrix}.$$

Matlab codes:

$$X = [-5.5, 6.8; -4.5, 5.2; -3.3, 4.6; -2.5, 3.2; -1.3, 2.8; -0.5, 2.1; 0.2, 1.9; 1.5, 1.6; 2.8, 1.3; 3.3, 0.8];$$

% Test data

$$Y_t = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix};$$

```
P = [ones(10,1), X(:,1), X(:,2), X(:,1).^2, X(:,1).*X(:,2),  
X(:,2).^2];
```

% Training

% Prediction

$$Y_{ts_est} = P_t * W$$

Binary Learning:

- For the case of **under-determined systems** with $m < D + 1$, the covariance matrix $\mathbf{P}^T \mathbf{P} \in \mathbb{R}^{(D+1) \times (D+1)}$ is singular since $\mathbf{P}$ has at most m independent rows. In other words, the rank of $\mathbf{P}$ is at most m and this leads to difficulty in estimating $\mathbf{w}$ using (5.6) or (5.18).
- A practical way to handle the situation is by **restricting the feasible solution set** to limit the effective size of parameters to less than or equal to the data sample size m . This can be achieved by mapping the original parameters from the larger $\mathbb{R}^{D+1}$ space to the smaller $\mathbb{R}^m$ space using

$$\mathbf{w} = \mathbf{P}^T \boldsymbol{\alpha}, \quad (5.21)$$

where $\alpha \in \mathbb{R}^m$.

$$\min_{\mathbf{w}} J_{\text{SSE}}(\mathbf{w}) = \min_{\mathbf{w}} (\mathbf{y} - \mathbf{P}\mathbf{w})^T (\mathbf{y} - \mathbf{P}\mathbf{w}) \quad (5.22)$$
$$\min_{\alpha} J_{\text{SSE}}(\alpha) = \min_{\alpha} (\mathbf{y} - \mathbf{P}\mathbf{P}^T \alpha)^T (\mathbf{y} - \mathbf{P}\mathbf{P}^T \alpha). \quad (5.23)$$
$$\begin{aligned} -\mathbf{P}\mathbf{P}^T(\mathbf{y} - \mathbf{P}\mathbf{P}^T\alpha) &= \mathbf{0} \\ \Rightarrow \hat{\alpha} &= (\mathbf{P}\mathbf{P}^T)^{-1}\mathbf{y}, \end{aligned} \quad (5.24)$$
$$\hat{\mathbf{w}} = \mathbf{P}^T (\mathbf{P}\mathbf{P}^T)^{-1} \mathbf{v}. \quad (5.25)$$

$$\min_{\mathbf{w}} \|\mathbf{w}\|_2^2 \quad \text{subject to } \mathbf{y} - \mathbf{P}\mathbf{w} = \mathbf{0}, \quad (5.26)$$
$$\min_{\mathbf{w}} \|\mathbf{w}\|_2^2 + (\mathbf{y} - \mathbf{P}\mathbf{w})^T \boldsymbol{\alpha}, \quad (5.27)$$
$$\begin{aligned} 2\mathbf{w} - \mathbf{P}^T \alpha &= \mathbf{0} \\ \mathbf{w} &= \frac{1}{2} \mathbf{P}^T \alpha, \end{aligned} \quad (5.28)$$
$$\begin{aligned} \mathbf{y} &= \mathbf{P} \left(\frac{1}{2} \mathbf{P}^T \boldsymbol{\alpha} \right) \\ \Rightarrow \hat{\boldsymbol{\alpha}} &= 2(\mathbf{P}\mathbf{P}^T)^{-1} \mathbf{y}, \end{aligned} \quad (5.29)$$

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍 ↻

$$\hat{\mathbf{W}} = \mathbf{P}^T (\mathbf{P}\mathbf{P}^T)^{-1} \mathbf{Y}, \quad (5.30)$$

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

Given the following samples of data features (x_1, x_2) and their corresponding target outputs (y_1, y_2, y_3) for training.

x_1 :	-3.3	-2.5	-1.3	0.2	2.8	3.3
x_2 :	4.6	3.2	2.8	1.9	1.3	0.8
y_1 :	1	1	0	0	0	0
y_2 :	0	0	1	1	0	0
y_3 :	0	0	0	0	1	1

We want to predict the outputs of $(x_1, x_2) = (-1, 3.3)$ and $(x_1, x_2) = (2.5, 0.5)$ using a full polynomial model of third order.

Example 5.2(a) Cont'd

Similar to the previous example, the first step is to pack the data in matrix form:

$$\mathbf{X} = \begin{bmatrix} -3.3 & 4.6 \\ -2.5 & 3.2 \\ -1.3 & 2.8 \\ 0.2 & 1.9 \\ 2.8 & 1.3 \\ 3.3 & 0.8 \end{bmatrix}, \quad \mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

Python codes:

27 / 58

- With the goal of **shrinking the parametric weight values** ($\mathbf{w}$) during estimation to avoid over amplification of the output ($g(\mathbf{w}, \mathbf{x}_i) = \mathbf{w}^T \mathbf{p}(\mathbf{x}_i)$) where instability may occur, the error objective function in (5.3) can be modified to include a **weight-decay** term $\|\mathbf{w}\|_2^2$ giving

$$\begin{aligned} J_{\text{SSEr}}(\mathbf{w}) &= \|\mathbf{y} - \mathbf{P}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_2^2 \\ &= (\mathbf{y} - \mathbf{P}\mathbf{w})^T (\mathbf{y} - \mathbf{P}\mathbf{w}) + \lambda \mathbf{w}^T \mathbf{w}, \end{aligned} \quad (5.31)$$

where a scalar regularization constant λ is used to control the importance of $\|\mathbf{w}\|_2^2$ during minimization.

- The solution to minimizing this regularized cost function by putting its first-order derivative to zero ($\nabla_{\mathbf{w}} J_{\text{SSEr}}(\mathbf{w}) = \mathbf{0}$) is given by

$$\hat{\mathbf{w}} = (\mathbf{P}^T \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^T \mathbf{y}. \quad (5.32)$$

- We observe here that the weight-decay term appears in the solution as a scaled identity matrix within the matrix inverse term and this stabilizes the inversion even when the covariance $\mathbf{P}^T \mathbf{P}$ is singular in theory. This formulation is known as *ridge regression* or *weight-decay regularization*.
- In practice, the ridge regression applies well for over-determined systems when the size of training data samples is larger than the size of projection dimension, i.e., when $m > D + 1$ (recall that $\mathbf{P} \in \mathbb{R}^{m \times (D+1)}$ in (5.4)). However, the situation may become numerically ill-conditioned for under-determined systems when $m < D + 1$ even with weight-decay regularization. Particularly, the prediction may become very inaccurate.

$$J_{\text{SSer}}(\mathbf{W}) = \text{trace}\{(\mathbf{Y} - \mathbf{P}\mathbf{W})^T(\mathbf{Y} - \mathbf{P}\mathbf{W}) + \lambda \mathbf{W}^T \mathbf{W}\}, \quad (5.33)$$
$$\hat{\mathbf{W}} = (\mathbf{P}^T \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^T \mathbf{Y}. \quad (5.34)$$

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

The over-determined system of Example 5.1(a) is next trained with primal ridge regression using (5.34) to give

$$\hat{\mathbf{W}} = (\mathbf{P}^T \mathbf{P} + 0.001 \mathbf{I})^{-1} \mathbf{P}^T \mathbf{Y} = \begin{bmatrix} 0.2371 & -0.1819 & 0.8951 \\ -0.8396 & 1.7396 & -0.8742 \\ -0.4231 & 1.5058 & -1.0415 \\ 0.1665 & -0.3811 & 0.2114 \\ 0.3408 & -0.8306 & 0.4795 \\ 0.1457 & -0.4333 & 0.2795 \end{bmatrix}.$$

The test output is

$$\hat{\mathbf{G}} = \mathbf{P}_t \hat{\mathbf{W}} = \begin{bmatrix} 0.3085 & 0.6886 & 0.0053 \\ -0.5709 & 1.3915 & 0.1792 \end{bmatrix}.$$

Example 5.1(b) Cont'd

Matlab codes:

```
% Training ridge regression
```

```
W = inv(P'*P+0.001*eye(6))*P'*Y;
```

```
% Prediction
```

```
Yts_est = Pt*W
```

Dual Ridge Regression

Binary Learning: An alternative solution to the above ridge regression is to perform the inverse in the smallest space of the two possibilities, i.e., either in the space of the feature dimension ($D + 1$) or in the space of data sample size dimension (m). This alternative approach is called the **dual ridge regression** and is given by the following manipulation:

$$\begin{aligned}(\mathbf{P}^T \mathbf{P} + \lambda \mathbf{I}) \mathbf{w} &= \mathbf{P}^T \mathbf{y} \\ \Rightarrow \mathbf{w} &= \lambda^{-1} (\mathbf{P}^T \mathbf{y} - \mathbf{P}^T \mathbf{P} \mathbf{w}) \\ \Rightarrow \mathbf{w} &= \lambda^{-1} \mathbf{P}^T (\mathbf{y} - \mathbf{P} \mathbf{w}) \\ \Rightarrow \mathbf{w} &= \mathbf{P}^T \boldsymbol{\alpha},\end{aligned}\tag{5.35}$$

Dual Ridge Regression

where

$$\begin{aligned}
 \alpha &= \lambda^{-1}(\mathbf{y} - \mathbf{P}\mathbf{w}) \\
 \Rightarrow \lambda\alpha &= (\mathbf{y} - \mathbf{P}\mathbf{w}) \\
 \Rightarrow \lambda\alpha &= (\mathbf{y} - \mathbf{P}\mathbf{P}^T\alpha) \\
 \Rightarrow \mathbf{P}\mathbf{P}^T\alpha + \lambda\alpha &= \mathbf{y} \\
 \Rightarrow (\mathbf{P}\mathbf{P}^T + \lambda\mathbf{I})\alpha &= \mathbf{y} \\
 \Rightarrow \hat{\alpha} &= (\mathbf{P}\mathbf{P}^T + \lambda\mathbf{I})^{-1}\mathbf{y} \\
 \Rightarrow \hat{\alpha} &= (\mathbf{K} + \lambda\mathbf{I})^{-1}\mathbf{y}. \tag{5.36}
 \end{aligned}$$

Dual Ridge Regression

The term $\mathbf{K} = \mathbf{P}\mathbf{P}^T$ is known as the Gram matrix. Notice that $\hat{\mathbf{w}} = \mathbf{P}^T \hat{\boldsymbol{\alpha}}$ and each **trained output** can be obtained as

$$\begin{aligned} g(\mathbf{x}_i, \hat{\mathbf{w}}) &= \hat{\mathbf{w}}^T \mathbf{p}(\mathbf{x}_i) \\ &= (\mathbf{P}^T \hat{\boldsymbol{\alpha}})^T \mathbf{p}(\mathbf{x}_i) \\ &= \hat{\boldsymbol{\alpha}}^T \mathbf{P} \mathbf{p}(\mathbf{x}_i), \quad i = 1, 2, \dots, m. \end{aligned} \tag{5.37}$$

Recall that $\mathbf{P}$ is a function of training inputs $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m]^T$ (see (5.4)), the **entire set of trained outputs can also be expressed in stacked matrix form** as follows:

$$\begin{aligned} \mathbf{g}(\mathbf{X}, \hat{\mathbf{w}}) &= \mathbf{P} \hat{\mathbf{w}} \\ &= \mathbf{P} \mathbf{P}^T \hat{\boldsymbol{\alpha}}. \end{aligned} \tag{5.38}$$

$$\begin{aligned} g(\mathbf{x}_j, \hat{\mathbf{w}}) &= \hat{\mathbf{w}}^T \mathbf{p}(\mathbf{x}_j) \\ &= (\mathbf{P}^T \hat{\boldsymbol{\alpha}})^T \mathbf{p}(\mathbf{x}_j) \\ &= \hat{\boldsymbol{\alpha}}^T \mathbf{P} \mathbf{p}(\mathbf{x}_j), \quad j = 1, 2, \dots, n. \end{aligned} \quad (5.39)$$

In a similar way, by indicating the set of unseen test data samples with a subscript t , the **test prediction output can be stacked** like those in training case as follows:

$$\begin{aligned} \mathbf{g}(\mathbf{X}_t, \hat{\mathbf{w}}) &= \mathbf{P}_t \hat{\mathbf{w}} \\ &= \mathbf{P}_t \mathbf{P}^T \hat{\boldsymbol{\alpha}}. \end{aligned} \quad (5.40)$$

$$\hat{\mathbf{W}} = \mathbf{P}^T \hat{\mathbf{A}}, \quad (5.41)$$

where $\hat{\mathbf{A}}$ is estimated via

The resulted training and test estimation outputs are respectively

and

For classification, a decision making process is followed.

Example 5.2(b)

The under-determined system of Example 5.2(a) is next trained with dual ridge regression using (5.41) and (5.42) with $\lambda = 0.001$ to give

$$\hat{\mathbf{W}} = \mathbf{P}^T \hat{\mathbf{A}} = \mathbf{P}^T (\mathbf{P}\mathbf{P}^T + 0.001\mathbf{I})^{-1} \mathbf{Y}$$

$$= \begin{bmatrix} 0.096921 & 0.20029239 & -0.06474344 \\ 0.07552804 & 0.19689304 & -0.06823502 \\ 0.09613258 & 0.1879961 & -0.05108843 \\ 0.09031235 & 0.08402066 & -0.02260897 \\ 0.03361644 & 0.16115132 & -0.04479921 \\ 0.05586837 & 0.09662081 & -0.00547768 \\ -0.0348168 & -0.02732227 & 0.02566001 \\ -0.01114985 & -0.14704293 & 0.07028742 \\ -0.07025671 & -0.06801809 & 0.07231998 \\ -0.06722926 & 0.00725292 & 0.022682 \end{bmatrix}.$$

By utilizing the transformed text matrix $\mathbf{P}_t$, the test output is computed as

$$\hat{\mathbf{G}} = \mathbf{P}_t \hat{\mathbf{W}} = \begin{bmatrix} -0.7265 & 1.7717 & 0.1343 \\ 0.3231 & 0.6092 & 0.2091 \end{bmatrix}.$$

Python codes (see Example 5.2(a) for the formation of matrices $\mathbf{P}$, $\mathbf{P}_t$ and $\mathbf{Y}$):

```
W2 = P.T @ inv(P @ P.T + Lambda*np.eye(P.shape[0])) @ Y
```

$$Y_t^{\text{predict2}} = P_t @ W_2$$

```
print(Yt_predict2)
```


Kernel Regression

Binary Learning: According to the representer theorem, the minimizer of any arbitrary cost function plus a strict monotonic increasing transformation of the minimizer under a normed RKHS admits a representation of the form as a linear combination of kernelized training data points.

In other words, the output g of a projected test data point $\mathbf{p} := \mathbf{p}(\mathbf{x})$ can be expressed in terms of the given set of projected training data $\mathbf{p}_i := \mathbf{p}(\mathbf{x}_i)$, $i = 1, \dots, m$ in inner product form to fit the target y with an arbitrary error ϵ to give

$$\begin{aligned} y &= g(\mathbf{p}, \mathbf{w}) + \epsilon \\ &= \mathbf{p}^T \mathbf{w} + \epsilon \\ &= \sum_{i=1}^m \alpha_i k(\mathbf{p}, \mathbf{p}_i) + \epsilon, \end{aligned} \tag{5.45}$$

with a kernel function defined by $k(\mathbf{p}, \mathbf{p}_i) = \langle \mathbf{p}, \mathbf{p}_i \rangle$ instead of a parametric form governed by $\mathbf{w} \in \mathbb{R}^{(D+1)}$.

Note: Recall from eqns (3.57)-(3.59) of L3 that the feature mappings $\mathbf{p}_i$ and $\mathbf{p}$ need not be explicitly computed. A prominent example is the polynomial kernel, $k(\mathbf{x}_i, \mathbf{x}) = (1 + \langle \mathbf{x}_i, \mathbf{x} \rangle)^r$, where the monomials remain implicit. By utilizing this form, one avoids the high-dimensional expansion of the feature space, making $k(\mathbf{x}_i, \mathbf{x}) = (1 + \langle \mathbf{x}_i, \mathbf{x} \rangle)^r$ the preferred formulation for polynomial kernels.

Kernel Regression

Collectively, for m number of training points stacked as a matrix

$\mathbf{P} = [\mathbf{p}_1, \dots, \mathbf{p}_m]^T \in \mathbb{R}^{m \times (D+1)}$, and a target vector $\mathbf{y} = [y_1, \dots, y_m]^T \in \mathbb{R}^m$ with errors $\boldsymbol{\epsilon} = [\epsilon_1, \dots, \epsilon_m]^T \in \mathbb{R}^m$ we can write

$$\begin{aligned} \mathbf{y} &= \mathbf{P}\mathbf{w} + \boldsymbol{\epsilon} \\ &= \mathbf{P}\mathbf{P}^T \boldsymbol{\alpha} + \boldsymbol{\epsilon} \\ &= \mathbf{K}\boldsymbol{\alpha} + \boldsymbol{\epsilon}, \end{aligned} \tag{5.46}$$

where

$$\mathbf{K} = \mathbf{P}\mathbf{P}^T, \quad \mathbf{w} = \mathbf{P}^T \boldsymbol{\alpha}, \tag{5.47}$$

with $\boldsymbol{\alpha} \in \mathbb{R}^m$ and $\mathbf{K} \in \mathbb{R}^{m \times m}$ is also known as the *training kernel matrix*. Similar to the estimation for under-determined systems, the representation by $\mathbf{w} = \mathbf{P}^T \boldsymbol{\alpha}$ can be considered a reparameterization from $\mathbf{w} \in \mathbb{R}^{(D+1)}$ in feature dimension to $\boldsymbol{\alpha} \in \mathbb{R}^m$ in sample dimension.

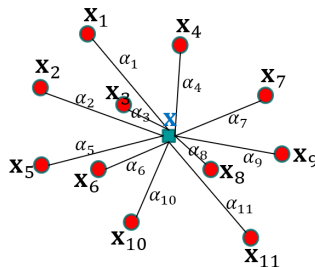
Question: Given that an explicit feature matrix $\mathbf{P}$ is not required for **polynomial kernels**, how can the kernel matrix $\mathbf{K}$ be efficiently computed in **matrix form**?

This shows that in the linear model, each data sample $\mathbf{x}$ can be expressed as a linear combination of training samples $\mathbf{x}_i, i = 1, \dots, m$ for some scalars $\alpha_i, i = 1, \dots, m$.

This brings us to the **Hilbert vector space**, where the similarity score of a pair of vectors $\mathbf{x}$ and $\mathbf{y}$ may be represented by the inner product associated with the space.

$$k(\mathbf{x}, \mathbf{y}) = \langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T \mathbf{y} = \mathbf{x} \cdot \mathbf{y}$$

$$\begin{aligned} f_{\mathbf{w}}(\mathbf{x}) &= \mathbf{w}^T \mathbf{x} \\ &= \sum_{i=1}^m \alpha_i k(\mathbf{x}_i, \mathbf{x}) \end{aligned}$$



Kernel Regression

The learning problem can be **formulated in kernel form to solve for the parameter α in the dual space** with respect to the given training data. Suppose $\mathbf{K}$ is non-singular and consider the following error function to be minimized:

$$J(\alpha) = \frac{1}{2} \epsilon^T \epsilon = \frac{1}{2} (\mathbf{y} - \mathbf{K}\alpha)^T (\mathbf{y} - \mathbf{K}\alpha). \quad (5.48)$$

By setting the first partial derivative of $J(\alpha)$ (5.48) with respect to α to zero, we have

$$\begin{aligned} \mathbf{K}^T (\mathbf{y} - \mathbf{K}\alpha) &= \mathbf{0} \\ \Rightarrow \mathbf{K}^T \mathbf{y} - \mathbf{K}^T \mathbf{K} \alpha &= \mathbf{0} \\ \Rightarrow \hat{\alpha} &= (\mathbf{K}^T \mathbf{K})^{-1} \mathbf{K}^T \mathbf{y} \\ \Rightarrow \hat{\alpha} &= \mathbf{K}^{-1} \mathbf{y}, \end{aligned} \quad (5.49)$$

since $\mathbf{K}$ is non-singular and $(\mathbf{K}^T \mathbf{K})^{-1} = \mathbf{K}^{-1} (\mathbf{K}^T)^{-1}$. Typically in practice, the inverse term in (5.49) exists for data with $m < D + 1$ such that $\mathbf{K}$ has full rank.

$$\begin{aligned} \mathbf{g}_t(\hat{\boldsymbol{\alpha}}, \mathbf{p}) &= \mathbf{P}_t \hat{\mathbf{w}} \\ &= \mathbf{P}_t \mathbf{P}^T \hat{\boldsymbol{\alpha}} \\ &= k(\mathbf{P}_t, \mathbf{P}) \mathbf{K}^{-1} \mathbf{y} \\ &= \mathbf{K}_t \mathbf{K}^{-1} \mathbf{y}. \end{aligned} \quad (5.50)$$

Remark: $\mathbf{K}_t = k(\mathbf{P}_t, \mathbf{P}) \in \mathbb{R}^{n \times m}$ is known as the *test kernel matrix*.

Kernel Ridge Regression

Binary Learning: From the kernel perspective, the parameter vector $\mathbf{w} \in \mathbb{R}^{D+1}$ can be projected onto a subspace $\alpha \in \mathbb{R}^m$ via

$$\mathbf{w} = \mathbf{P}^T \alpha, \quad (5.53)$$

when when $m < D + 1$ where we can write

$$\|\mathbf{w}\|_2^2 = \mathbf{w}^T \mathbf{w} = \alpha^T \mathbf{P} \mathbf{P}^T \alpha. \quad (5.54)$$

By defining $\mathbf{K} = \mathbf{P} \mathbf{P}^T = \mathbf{K}^T$, the regularized objective function can be written as

$$J(\alpha) = \frac{\lambda}{2} \alpha^T \mathbf{K}^T \alpha + \frac{1}{2} (\mathbf{y} - \mathbf{K} \alpha)^T (\mathbf{y} - \mathbf{K} \alpha), \quad (5.55)$$

where $\alpha \in \mathbb{R}^m$ corresponds to the parameter vector to be adjusted for minimizing J .

Kernel Ridge Regression

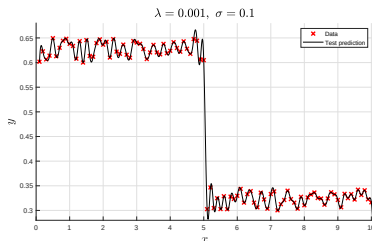
Assuming $\mathbf{K}$ is non-singular and by setting the first partial derivative of $J(\alpha)$ with respect to α to zero, we have

$$\begin{aligned}\lambda \mathbf{K}^T \alpha - \mathbf{K}^T (\mathbf{y} - \mathbf{K} \alpha) &= \mathbf{0} \\ \Rightarrow \lambda \mathbf{K}^T \alpha - \mathbf{K}^T \mathbf{y} + \mathbf{K}^T \mathbf{K} \alpha &= \mathbf{0} \\ \Rightarrow \lambda \mathbf{K}^T \alpha + \mathbf{K}^T \mathbf{K} \alpha &= \mathbf{K}^T \mathbf{y} \\ \Rightarrow \mathbf{K}^T (\mathbf{K} + \lambda \mathbf{I}) \alpha &= \mathbf{K}^T \mathbf{y} \\ \Rightarrow \hat{\alpha} &= (\mathbf{K} + \lambda \mathbf{I})^{-1} (\mathbf{K}^T)^{-1} \mathbf{K}^T \mathbf{y} \\ \Rightarrow \hat{\alpha} &= (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}. \quad (5.56)\end{aligned}$$

$$\begin{aligned}\hat{\mathbf{g}}(\mathbf{X}_t, \mathbf{w}) &= \mathbf{P}_t \mathbf{w} \\ &= \mathbf{P}_t \mathbf{P}^T \boldsymbol{\alpha} \\ &= k(\mathbf{P}_t, \mathbf{P})(\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}.\end{aligned}\tag{5.57}$$
$$\hat{\mathbf{G}}_t = k(\mathbf{P}_t, \mathbf{P})(\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{Y}. \quad (5.58)$$

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

Example 5.3 cont'd



```
% Trained output
```

```
y_tr = K*alpha;
```

```
% Form Test Kernel
```

```
x ts = [0.1:.01:10.05]; % Test data
```

```
for i = 1:length(x ts)
```

```
for i = 1:length(x)
```

```
Kts(i,j) = exp(-(x ts(i)-x(j))^2/(2*sigma^2));
```

end

end

```
% Test output
```

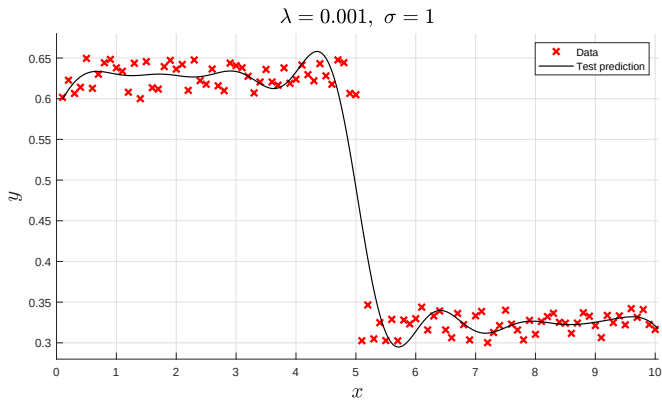
```
y_ts = Kts*alpha;
```

Prediction stage:

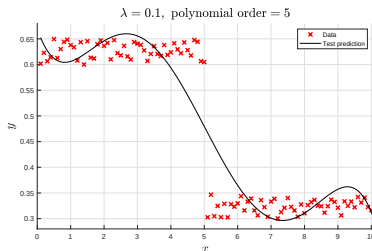
Trained output: $\hat{y}_{tr} = \mathbf{K}\hat{\alpha}$

Form the test kernel matrix: $\mathbf{K}_{ts}(i, j) = e^{\frac{1}{2\sigma^2} \|x_i^{ts} - x_j\|^2}$

Test output: $\hat{y}_{ts} = \mathbf{K}_{ts} \hat{\alpha}$



Use the kernel method to fit the data points shown below.



```
order = 5;
```

$$X = x';$$

```
K = (1+X*X').^order;
```

% Training

```
lambda = 0.1;
```

```
alpha = inv(K + lambda*eye(size(K)))*y;
```

```
% Trained output
```

```
y_tr = K*alpha;
```

```
% Form Test Kernel
```

```
x_ts = [0.1:.01:10.05]; % Test data
```

$$X_{ts} = x \ t s';$$

```
Kts = (1+Xts*X').^order;
```

```
% Test output
```

```
y_ts = Kts*alpha;
```

Training stage:

Form the kernel matrix: $\mathbf{K}(i, j) = (\mathbf{1} + \mathbf{X}\mathbf{X}^T)^{\circ 5}$

Compute the *dual coefficient vector* α : $\hat{\alpha} = (\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}$

Remark: The operator $(\cdot)^{\circ 5}$ denotes raising each element within $(\cdot)$ to the fifth power.

The symbol $\mathbf{1}$ denotes an $m \times m$ matrix of ones, where m represents the sample size.

Binary Classification: For two-category or two-class problems, a single-dimensional output $g(\mathbf{x}, \mathbf{w}) \in \mathbb{R}$ can be used to represent the desired target values of $y \in \{0, 1\} \in \mathbb{N}$ (or $y \in \{-1, +1\} \in \mathbb{N}$) for the two classes. A **decision threshold** τ can then be applied to $g(\mathbf{x}, \mathbf{w})$ to decide whether an input $\mathbf{x}$ belongs to “class-0” (or class “-1”) or “class-1” (or class “+1”):

$$cls(g(\mathbf{x}, \mathbf{w})) = \begin{cases} 1 & \text{if } g(\mathbf{x}, \mathbf{w}) \geq \tau \\ 0 & \text{if } g(\mathbf{x}, \mathbf{w}) < \tau \end{cases} . \quad (5.59)$$

For normalized distribution of $g(\mathbf{x}, \mathbf{w}) \in [0, 1]$ or $g(\mathbf{x}, \mathbf{w}) \in [-1, +1]$, this threshold is often chosen as $\tau = 0.5$ for $y \in \{0, 1\}$ or $\tau = 0$ for $y \in \{-1, +1\}$ respectively.

$$\hat{\mathbf{G}}_t = \begin{bmatrix} \mathbf{g}^T(\mathbf{x}_1, \mathbf{W}) \\ \mathbf{g}^T(\mathbf{x}_2, \mathbf{W}) \\ \vdots \\ \mathbf{g}^T(\mathbf{x}_n, \mathbf{W}) \end{bmatrix} = \begin{bmatrix} g(\mathbf{x}_1, \mathbf{w}_1) & g(\mathbf{x}_1, \mathbf{w}_2) & \cdots & g(\mathbf{x}_1, \mathbf{w}_C) \\ g(\mathbf{x}_2, \mathbf{w}_1) & g(\mathbf{x}_2, \mathbf{w}_2) & \cdots & g(\mathbf{x}_2, \mathbf{w}_C) \\ \vdots & \dots & \ddots & \vdots \\ g(\mathbf{x}_n, \mathbf{w}_1) & g(\mathbf{x}_n, \mathbf{w}_2) & \cdots & g(\mathbf{x}_n, \mathbf{w}_C) \end{bmatrix}. \quad (5.60)$$
$$cls(\mathbf{g}(\mathbf{x}_j, \mathbf{W})) = \arg \max_k g(\mathbf{x}_j, \mathbf{w}_k), \forall k = 1, 2, \dots, C. \quad (5.61)$$

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

The over-determined system of Example 5.1(b) has been trained with primal ridge regression. For category prediction, (5.61) is utilized to give the classification decision.

Example 5.1(c)

Matlab codes:

```
Yts_est_class = zeros(2,3);
for k=1:2
    [M,I] = max(Yts_est_b(k,:));
    Yts_est_class(k,I) = 1;
end
Yts_est_class =
    0 1 0
    0 0 1
```


The under-determined system of Example 5.2(b) has been trained with dual ridge regression. For category prediction, (5.61) is utilized to give the classification decision.

Example 5.2(c))

Python codes (see Example 2(a) and (b) for declarations and `Yt_predict2`):

```
## Class prediction
Yt_class_predict = np.zeros((2,3))
for k in range(2):
    idx = np.argmax(Yt_predict2[k,])
    Yt_class_predict[k,idx] = 1
print(Yt_class_predict)
[[0.  1.  0.]
 [0.  0.  1.]]
```

- Regression error based learning methods (essentially use of the squared error loss)
 - * Minimize the SSE cost for over-determined systems
 - * Minimize the SSE cost for under-determined systems
 - * Primal ridge regression
 - * Dual ridge regression
 - * Kernel regression
 - * Kernel ridge regression
- Classification by thresholding learning model output

Thank you!